

TwinCAT 4026, Programming Conveyor 01 and Stop 01.

In this chapter we will program the code to automate the Conveyor 01 and Stop 01 from the Model. We have different sensors and actuators on this section.

List of sensors:

- Photo Eye Conveyor 1.
- Stop1 Extend
- Stop1 Part Present
- Stop1 Return
- RFID1 Tag Present

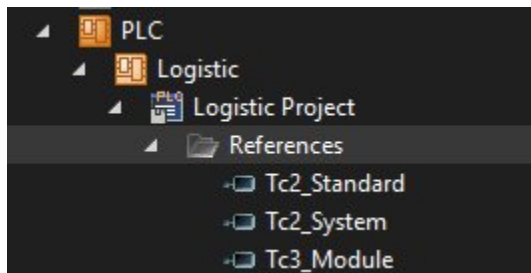
List of actuators:

- Conveyor 01
- Stop1 Return
- Request to read RFID1.

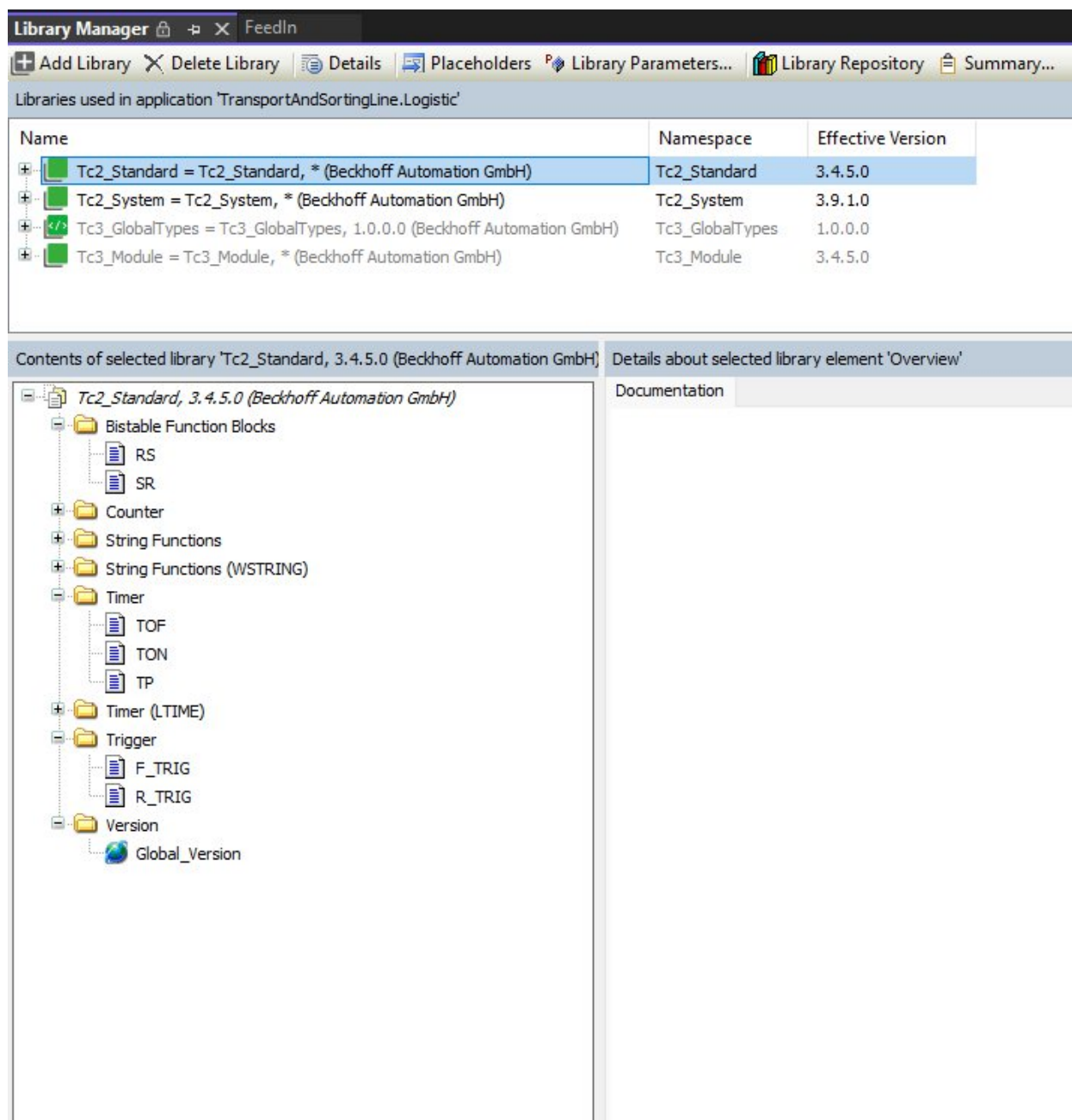
TwinCAT Libraries

We will be using different functions from TwinCAT standard library. Called TC2_Standard. The library was added when the project was created, remember we selected a standard PLC project.

The libraries used on the project are registered on under PLC >> References.



Standard PLC project include by default these three libraries. You can open the libraries to see the contents of each one. In this case we will focus on TC2_Standard. This library has function blocks to program timers, counters, rising and falling trigger bits, set/reset bits, string functions. For this exercise we will be using Timers and Trigger function blocks. We will use also the assignation which we used on the previous chapter.

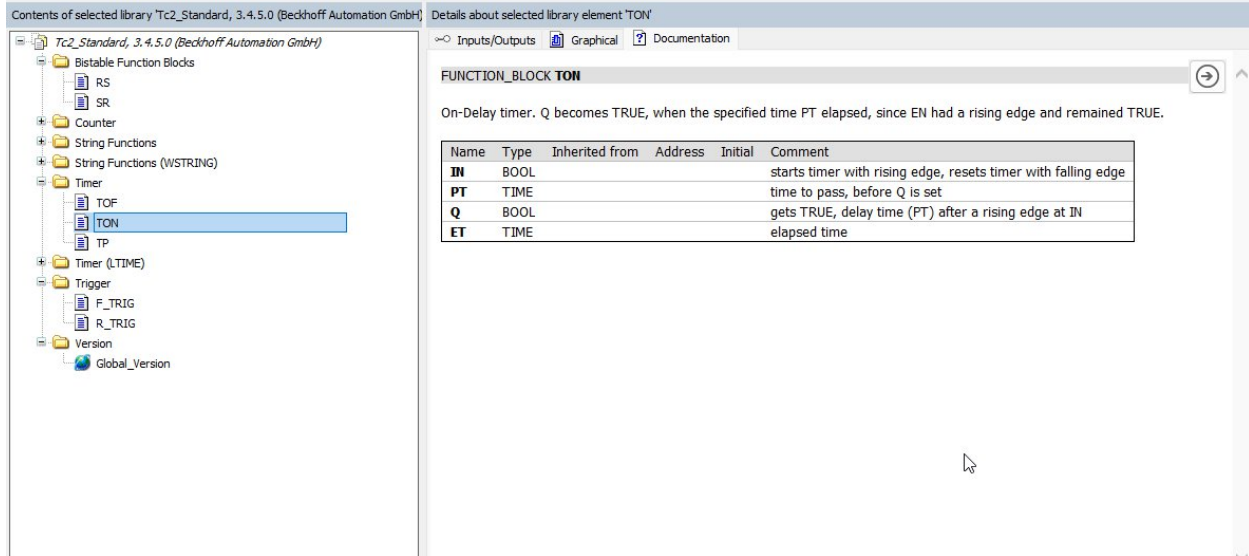


We need to study for this exercise how the Timer TON and the Trigger R_Trig function blocks works. They are other versions of timers and triggers but they work very similar.

TwinCAT Timer TON Function Block

The next picture describe how the TON timer works. It also describe the different parameters which are embeded on it.

Chapter 3 – TwinCAT 4026, Programming Conveyor 01 and Stop 01.



In the Inputs/Outputs sections we can see which signals are inputs and outputs and the purpose of them.

FUNCTION_BLOCK TON						
	Name	Type	Inheri...	Address	Initial	Comment
INPUT	IN	BOOL				starts timer with rising edge, reset...
INPUT	PT	TIME				time to pass, before Q is set
OUTPUT	Q	BOOL				gets TRUE, delay time (PT) after a ...
OUTPUT	ET	TIME				elapsed time

Use of Timer TON on the Code

In order to use a Timer, it is required to be added as a variable on the POU. Any function block consists of two parts: Instructions and Memory. When adding a timer in any POU we are reserving the memory and identifying the timer on the program.

The next picture shows the most typical way to add a timer on the section variables of the POU.

```
// FeedIn
i_PE_TMR:TON; //Timer to detect part on the PE
```

In this case we are using the name `i_PE_TMR` to name a TON timer. The intention of this name is to indicate in which part of the program is located. For example, the letter "i" means that the timer is inside the POU. The letters "PE" means it is a Photo Eye and the last term "TMR" means it is a timer. In this case, the meaning is an internal timer which checks the Photo Eye sensor.

The next picture shows how we can call this timer on the code.

```
//Call timer for detect part entering the system at PE
i_PE_TMR(IN:= PE_PartIn, PT:=T#500MS);
```

Chapter 3 – TwinCAT 4026, Programming Conveyor 01 and Stop 01.

We can see that our timer is going to start count when the PE_PartIn variable is true. The timer will be output the Q variable when it reach 500 mili-seconds. The time data type is indicate with the letter “T#”. These two letters indicate time.

We can use the Q variable from this time at any section of the code. For example the next picture show how we can use it.

```
//Part Intransit In
i_FeedInTransit:= (i_PE_TMR.Q AND MTR_On)
```

TwinCAT Trigger R_Trig Function Block

The next picture describe how the Raising Trigger (R_TRIG) works. It has only two parameters, Clk (Clock) and Q (output). This function is to check if the signal turn on. The Q will turn on only when the signal turn on and it will be on only one scan.

The screenshot shows the TwinCAT software interface. At the top, there are tabs for 'Add Library', 'Delete Library', 'Details', 'Placeholders', 'Library Parameters...', 'Library Repository', and 'Summary...'. Below these tabs, a table lists libraries used in the application 'TransportAndSortingLine.Logistic'.

Name	Namespace	Effective Version
Tc2_Standard = Tc2_Standard, * (Beckhoff Automation GmbH)	Tc2_Standard	3.4.5.0
Tc3_GlobalTypes = Tc3_GlobalTypes, 1.0.0.0 (Beckhoff Automation GmbH)	Tc3_GlobalTypes	1.0.0.0
Tc2_System = Tc2_System, * (Beckhoff Automation GmbH)	Tc2_System	3.9.1.0
Tc3_GlobalTypes = Tc3_GlobalTypes, 1.0.0.0 (Beckhoff Automation GmbH)	Tc3_GlobalTypes	1.0.0.0
Tc3_Module = Tc3_Module, * (Beckhoff Automation GmbH)	Tc3_Module	3.4.5.0

Below the table, there are two panels. The left panel shows the 'Contents of selected library 'Tc2_Standard, 3.4.5.0 (Beckhoff Automation GmbH)'. It lists various function blocks: Bistable Function Blocks, Counter, String Functions, String Functions (WSTRING), Timer, Timer (LTIME), Trigger, F_TRIG, R_TRIG, and Version. The right panel shows the 'Details about selected library element 'R_TRIG''. It has tabs for 'Inputs/Outputs', 'Graphical', and 'Documentation'. The 'Inputs/Outputs' tab is selected, showing the 'FUNCTION_BLOCK R_TRIG' with the description 'Rising Edge detection.' and a table of inputs and outputs.

Name	Type	Inherited from	Address	Initial	Comment
CLK	BOOL				Signal to detect
Q	BOOL				rising edge at signal detected

It has only one input and one output.

The screenshot shows the TwinCAT software interface. At the top, there are tabs for 'Inputs/Outputs', 'Graphical', and 'Documentation'. The 'Inputs/Outputs' tab is selected, showing the 'FUNCTION_BLOCK R_TRIG' with the description 'Rising Edge detection.' and a table of inputs and outputs.

	Name	Type	Inheri...	Address	Initial	Comment
INPUT	CLK	BOOL				Signal to detect
OUTPUT	Q	BOOL				rising edge at signal det...

The CLK signal is the signal to determine if it turn on. Q will turn on only when the signal has turn on, only for one scan cycle.

Use of R_TRIG on the Code

In order to use a R_Trigger, it is required to add a variable on the POU same as we did before with the Timer.

The next picture shows how to add a raising trigger on the code.

```
//Stop  
i_PartArrivesAtStop: R_TRIG;
```

We followed the same concept the letter “i” means that is internal raising trigger to the POU in use.

The next picture shows how we can call the raising trigger variable on the code.

```
//Part at Stop  
i_PartArrivesAtStop(CLK:=PRX_PartAtStop); //Used to kill the i_FeedInTransit
```

The only input to the raising trigger is the CLK parameter. We can use the Q variable on any section of the code, like is shown on the next picture.

```
//Part Intransit In  
i_FeedInTransit:= (i_PE_TMR.Q AND MTR_On) OR (i_FeedIntransit AND NOT i_PartArrivesAtStop.Q);
```

General information regarding the Function Blocks.

A function block has two parts:

- Memory
- Instructions

Memory is the area of the device where the instruction will save its internal information to determine the state. Instruction is the program that handles the use of that memory. This is why when we use any function block we need to add on the variable section and in the code section.

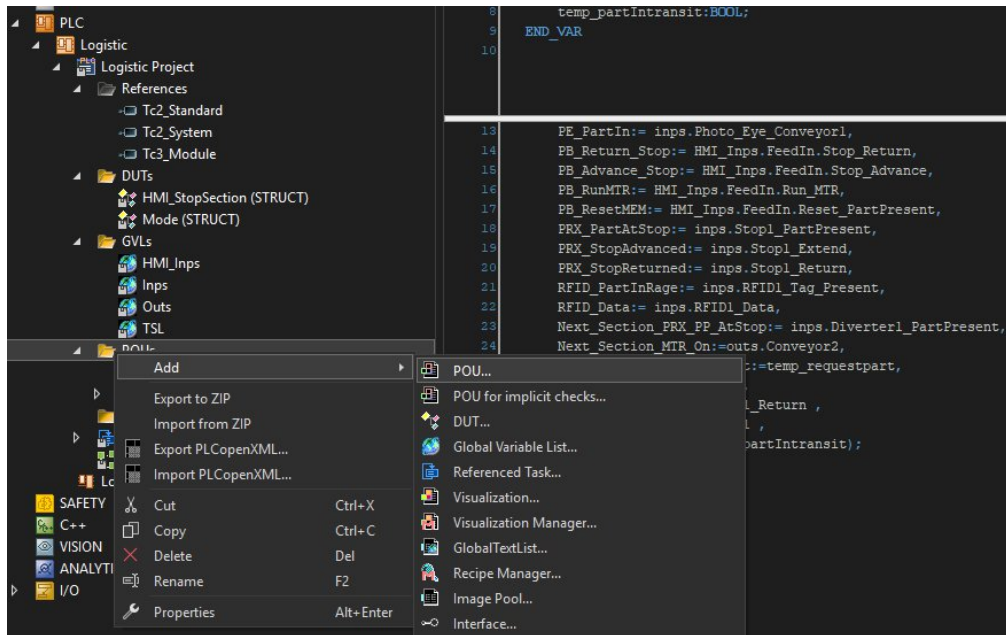
FeedIn Function Block

We need to automate the Conveyor01 and Stop01. The best way to do it is creating our own function block. This function block will have the same behavior as TwinCAT standard libraries. It will have inputs, outputs and internal variables. It is also possible to have InOut variables but we will not cover them.

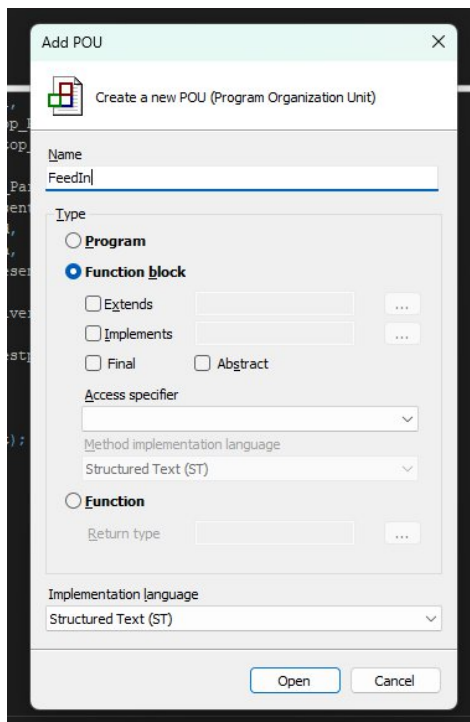
Chapter 3 – TwinCAT 4026, Programming Conveyor 01 and Stop 01.

The next step is to create a new Function Block.

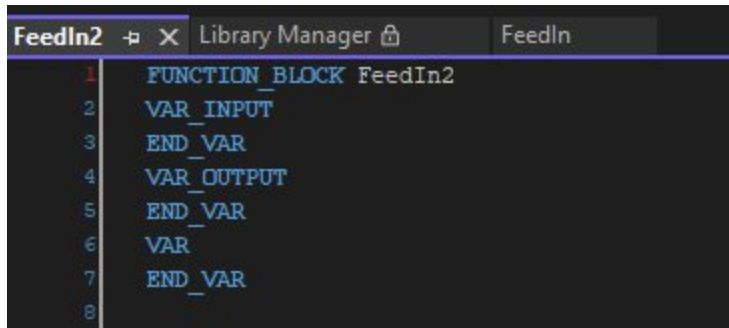
Right click on the POU folder, select add and click POU.



Select Function Block and type the name FeedIn



By default TwinCAT creates three spaces to declare different types of variables.



We are teaching by example. We ask to the students to review in very detail the code to understand how is programmed.

FeedIn Input variables.

VAR_INPUT

//Operation Mode

m_Auto:BOOL;	//m_Cycle - Auto and Cycle Running
m_Manual:BOOL;	//m_Manual - Manual Operation Only
m_NoMode:BOOL;	//m_NoMode - No Machine mode selected

//Manual Functions

PB_Return_Stop:BOOL;	//Push Button to Return Stop
PB_Advance_Stop:BOOL;	//Push Button to Advance Stop
PB_RunMTR:BOOL;	//Push button to Run Motor
PB_ResetMEM:BOOL;	//Push Button to Reset Memory of Part Present
PE_PartIn:BOOL;	//Photo Eye Part at beginning of Conveyor

//Prox Sensors

PRX_PartAtStop:BOOL;	//Prox Part at Stop
PRX_StopAdvanced:BOOL;	//Prox Stop at Advance Position
PRX_StopReturned:BOOL;	//Prox Stop at Return Position
RFID_PartInRage:BOOL;	//RFID Part in Range
RFID_Data:DINT;	//RFID Data

//Next Section

Next_Section_PRX_PP_AtStop:BOOL;	//Next Section part Present at Stop
----------------------------------	-------------------------------------

Chapter 3 – TwinCAT 4026, Programming Conveyor 01 and Stop 01.

```
Next_Section_MTR_On:BOOL;           //Next Section Motor On
Next_Section_RequestPart:BOOL;       //Next Section Station Request Part
END_VAR
```

FeedIn Output Variables.

VAR_OUTPUT

```
//Variables to Communicate with Actuators and Other Sections

MTR_On:BOOL;           //Motor On
STOP_Return:BOOL;      //Stop Returning
REQ_RFID:BOOL;         //Request to Read RFID
PartAtStopMem:BOOL;    //Part Preset at Stop Memory
PartInTransitOut:BOOL; //Part In Transit Out Memory
END_VAR
```

FeedIn Internal Variables.

VAR

```
//Internal Variables

//Condition to Return Stop
i_Stop_ClearToReturn:BOOL; //Clear to Return Stop (No Crash)
i_Stop_CondToAReturn:BOOL; //Auto to Return Stop (Auto Mode)

//Condition to Advance Stop
i_Stop_ClearToAdvance:BOOL; //Clear to Advance Stop (No Crash)
i_Stop_CondToAdvance:BOOL;  //Auto to Advance Stop (Auto Mode)

//Condition to Run Motor
i_MTR_ClearToRun:BOOL;      //Clear to Run Motor
i_MTR_CondToRun:BOOL;       //Auto to Run Motor

// FeedIn
i_PE_TMR:TON;               //Timer to detect part on the PE
```



```
i_FeedInTransit_Mem:BOOL;      //Part Intransit In

//Stop

i_PartArrivesAtStop_ONS: R_TRIG; //Part Arrived at Stop Rising Trigger (only one shot,
only one scan with true value).

i_PartNotAtStop_TMR:TON;      //Not Part present at Stop (Time On Delay Timer)

i_PartAtStop_TMR:TON;        //Part at Stop Present (Time On Delay Timer)

i_PartAtStop_TMR_ONS:R_Trig;  //Part Present At Stop Rising Trigger (only one shot,
only one scan with true value).

i_PartInTransit_ONS:R_Trig;   //Part In Transit One Shot Rising Trigger (only one
shot, only one scan with true value).

i_StopAdvance_Delay:TON;     //Stop Advance Delay TON

i_StopReturn_Delay:Ton;      //Stop Return Delay TON

//Next Section Part Present at Stop

i_Next_Section_PRX_PP_AtStop_ONS:R_TRIG; //Next Section Part Present at Stop
Rising Trigger (only one shot, only one scan with true value).

END_VAR

FeedIn Code.
//Call timer for detect part entering the system at PE

i_PE_TMR(IN:= PE_PartIn, PT:=T#500MS);

//Part Intransit In

i_FeedInTransit_Mem:= (i_PE_TMR.Q AND MTR_On) OR (i_FeedInTransit_Mem AND NOT
i_PartArrivesAtStop_Ons.Q);

//Part at Stop

i_PartArrivesAtStop_Ons(CLK:=PRX_PartAtStop); //Used to kill the i_FeedInTransit

//Detect a part has been advance all the way it on the stop

i_PartNotAtStop_TMR(in:=PRX_PartAtStop, PT:=T#100MS);

i_PartAtStop_TMR(In:=PRX_PartAtStop, PT:=T#1S);

i_PartAtStop_TMR_ONS(Clk:=i_PartAtStop_TMR.Q);

//Part at Stop Mem
```

```
PartAtStopMem:= i_PartAtStop_TMR_ONS.Q OR (PartAtStopMem AND NOT  
(i_PartInTransit_ONS.Q));
```

```
//Part Intransit
```

```
i_PartInTransit_ONS(Clk:=MTR_On AND PRX_StopReturned AND PartAtStopMem);
```

```
//PartIntransit
```

```
PartInTransitOut:= i_PartInTransit_ONS.Q OR (PartInTransitOut AND NOT  
i_Next_Section_PRX_PP_AtStop_ONS.Q);
```

```
//Next Section OneShot
```

```
i_Next_Section_PRX_PP_AtStop_ONS(Clk:=Next_Section_PRX_PP_AtStop);
```

```
//Stop Conditions to Open or Advance
```

```
i_StopAdvance_Delay(IN:=prx_StopAdvanced AND NOT Stop_Return, PT:=T#1S);
```

```
i_StopReturn_Delay(IN:=prx_StopReturned, PT:=T#1S);
```

```
//Conditions to Open Stop
```

```
i_Stop_ClearToReturn:= PartAtStopMem OR PartInTransitOut; //Crash Condition
```

```
i_Stop_CondToAReturn:= Next_Section_MTR_On AND i_Stop_ClearToReturn; //AutoCondition
```

```
//Conditions to Close Stop
```

```
i_Stop_ClearToAdvance:= i_PartNotAtStop_TMR.Q; //Crash Condition
```

```
i_Stop_CondToAdvance:= NOT PartInTransitOut AND i_Stop_ClearToAdvance AND NOT  
Next_Section_RequestPart; //AutoCondition
```

```
//Stop Condition to Return
```

```
STOP_Return:= (m_Auto AND( i_Stop_CondToAReturn AND NOT i_Stop_CondToAdvance))  
OR (m_Manual AND (PB_Return_Stop AND i_Stop_ClearToReturn) AND NOT  
(PB_Advance_Stop AND i_Stop_ClearToAdvance));
```

```
//Condition to Determine if the system is clear to start the conveyor
```

```
i_MTR_ClearToRun:= i_StopAdvance_Delay.Q OR i_StopReturn_Delay.Q;
```

```
i_MTR_CondToRun:= ((i_FeedInTransit_Mem OR PE_PartIn) AND (i_StopAdvance_Delay.Q))  
OR ((i_PartAtStop_TMR.In AND i_StopAdvance_Delay.Q AND NOT i_PartAtStop_TMR.Q) OR  
((i_PartAtStop_TMR.Q OR(PartInTransitOut)) AND i_StopReturn_Delay.Q)); //Out
```

```
MTR_On:= (m_Auto AND i_MTR_CondToRun) OR (m_Manual AND PB_RunMTR AND  
i_MTR_ClearToRun);
```

//Reset Part Present using HMI

IF (m_Manual AND PB_ResetMEM) THEN

 PartInTransitOut:=FALSE;

 PartAtStopMem:=FALSE;

END_IF;

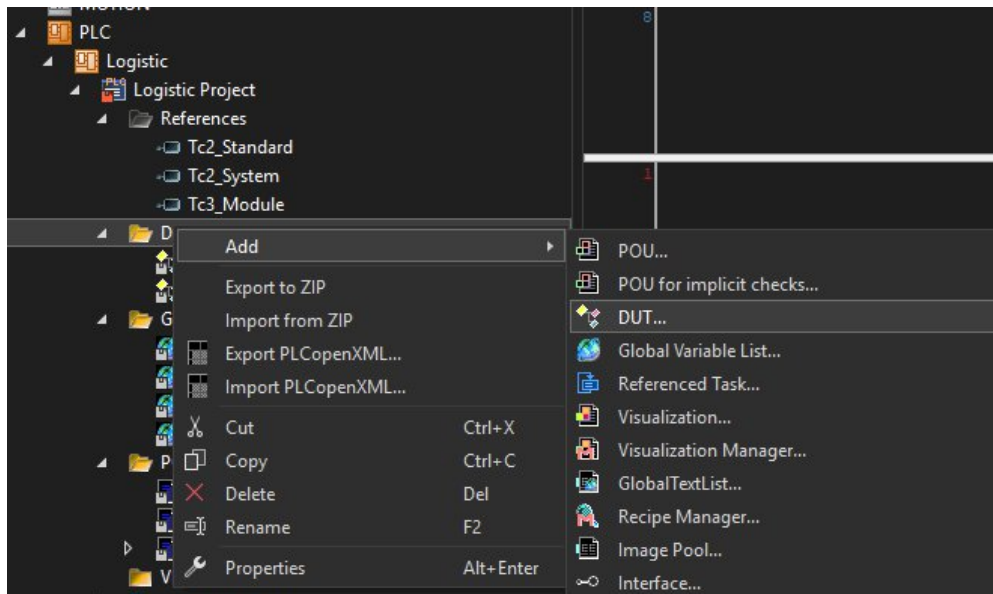
Data Unit Types (DUT).

DUT's are user defined data type. You can create your own data types that makes more sense for different functions. We are going to declare two DUTs.

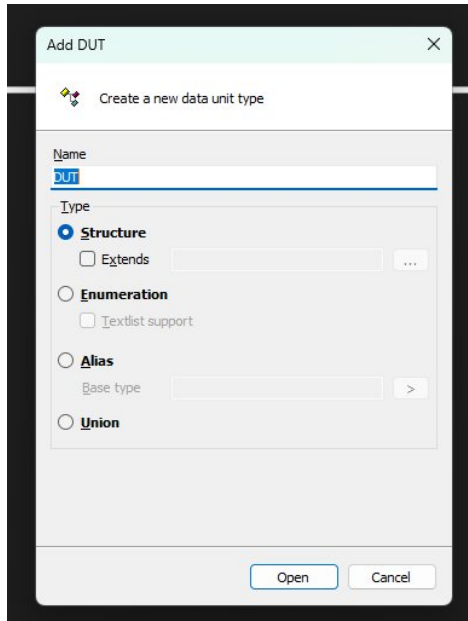
The first DUT is called m_Cycles. This data type will have signals related with the machine operation, for example; Auto, Manual and No Mode.

In order to add a DUT.

Right click the folder DUT >> ADD >> DUT.



Type the name of the DUT.



Click Open.

Create two DUT's one called m_Cycles and other called PZR_Commands.

m_Cycles DUT.

STRUCT

```
Auto:BOOL;  
Manual:BOOL;  
NoMode:BOOL;  
Cycle_On:Bool;
```

END_STRUCT

END_TYPE

PZR_Commands DUT.

STRUCT

```
PB_Return_Stop:BOOL;    //Push Button to Return Stop  
PB_Advance_Stop:BOOL;   //Push Button to Advance Stop  
PB_RunMTR:BOOL;         //Push button to Run Motor  
PB_ResetMEM:BOOL;       //Push Button to Reset Memory of Part Present
```

```
        PE_PartIn:BOOL;           //Photo Eye Part at beginning of Conveyor  
  
END_STRUCT  
  
END_TYPE
```

HMI Commands.

We need to create a new list on Global Variables List Called HMI_Inps. In the next chapters we will use this HMI_Inps GVL to program the Machine Interface. Remember to check the previous chapters if you need help to add Global Variable List.

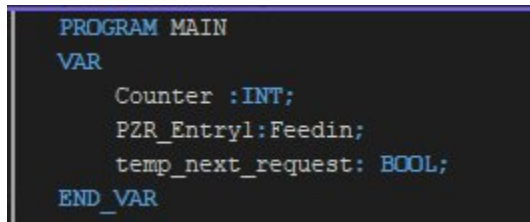
HMI_Inps GVL.

```
VAR_GLOBAL  
  
    m_Modes:m_Modes;  
  
    PB_PZR_Entry1:PZR_Commands;  
  
END_VAR
```

Call FeedIn Function Block.

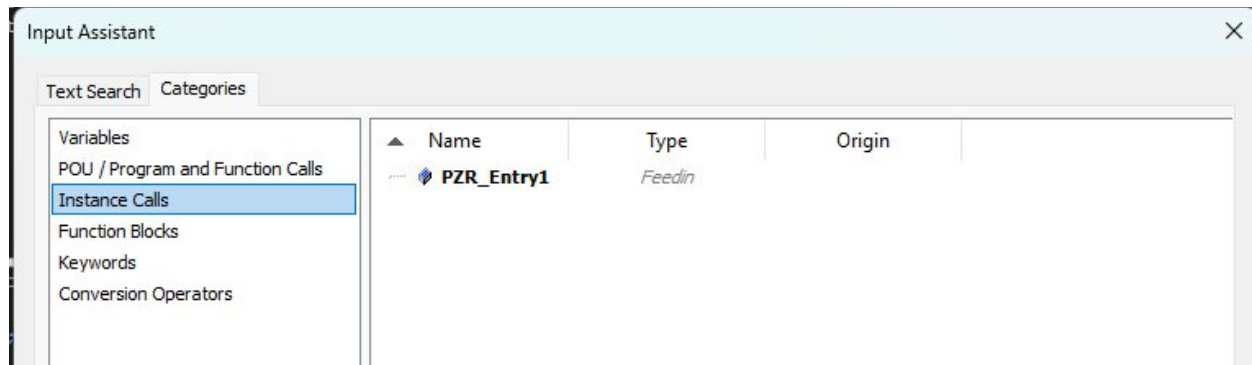
We need to add the Function Block FeedIn in the variable section on the Main POU. We will need also to add another variable to indicate next conveyor section is ready to receive a part, we called this variable temp_next_request.

The next picture show how to declare these variables.



```
PROGRAM MAIN  
VAR  
    Counter :INT;  
    PZR_Entry1:Feedin;  
    temp_next_request: BOOL;  
END_VAR
```

They are multiple ways to add FeedIn Function Block call on the code. The fastest way is to use the input assistant. Right click on any section on the code and select Input assistant or use F2 key. Select Instance calls and select the PZR_Entry1.



Once select the Function Block will be called with all the parameters.

```
//Call Main Functions

PZR_Entry1(
    m_Auto:= ,
    m_Manual:= ,
    m_NoMode:= ,
    PB_Return_Stop:= ,
    PB_Advance_Stop:= ,
    PB_RunMTR:= ,
    PB_ResetMEM:= ,
    PE_PartIn:= ,
    PRX_PartAtStop:= ,
    PRX_StopAdvanced:= ,
    PRX_StopReturned:= ,
    RFID_PartInRage:= ,
    RFID_Data:= ,
    Next_Section_PRX_PP_AtStop:= ,
    Next_Section_MTR_On:= ,
    Next_Section_RequestPart:= ,
    MTR_On=> ,
    STOP_Return=> ,
    REQ_RFID=> ,
    PartAtStopMem=> ,
    PartInTransitOut=> );
```

The next step is to assign these parameters.

Assignment of the parameters of the FeedIn Function Block call.

//Call Main Functions

```
PZR_Entry1(
    m_Auto:= HMI_Inps.m_Modes.Cycle_On,
    m_Manual:= HMI_Inps.m_Modes.Manual,
```

```
m_NoMode:= HMI_Inps.m_Modes.NoMode,
PB_Return_Stop:= HMI_Inps.PB_PZR_Entry1.PB_Return_Stop,
PB_Advance_Stop:= HMI_Inps.PB_PZR_Entry1.PB_Advance_Stop,
PB_RunMTR:= HMI_Inps.PB_PZR_Entry1.PB_RunMTR,
PB_ResetMEM:=HMI_Inps.PB_PZR_Entry1.PB_ResetMEM ,
PE_PartIn:= inps.Photo_Eye_Conveyor1,
PRX_PartAtStop:= inps.Stop1_PartPresent,
PRX_StopAdvanced:= inps.Stop1_Extend,
PRX_StopReturned:= inps.Stop1_Return,
RFID_PartInRage:= inps.RFID1_Tag_Present,
RFID_Data:= inps.RFID1_Data,
Next_Section_PRX_PP_AtStop:= inps.Diverter1_PartPresent,
Next_Section_MTR_On:= outs.Conveyor2,
Next_Section_RequestPart:= temp_next_request,
MTR_On=>outs.Conveyor1 ,
STOP_Return=> outs.Stop1_Return,
REQ_RFID=> outs.Req_RFID1,
PartAtStopMem=> ,
PartInTransitOut=> );
```

Debug the application.

In this section you will put in practice your knowledge from the previous chapter. Take in consideration that we need to force some variables.

Global Variable List:

HMI_Inps.

We need to force the Cycle_On and Auto.

Outs.

We need to force Conveyor2. This in order to be able to transfer all the parts out of the system.

POU Main.

We need to force the variable **temp_next_request**, in order to indicate to the FeedIn function block logic that the next section of conveyor is ready to receive the part.